

EXPERIMENT-3

SIDDHI SAMBHAV

(24UADS1070)

Objective :- WAP to implement a three-layer neural network using Tensor flow library (only, no keras) to classify MNIST handwritten digits dataset. Demonstrate the implementation of feed-forward and back-propagation approaches.

Description :-

A Backpropagation Neural Network (BPNN) is a supervised learning model that consists of multiple layers and is trained using gradient descent optimization.

The architecture generally includes:

- Input Layer
- One Hidden Layer (or more)
- Output Layer

In this experiment, we implement a **3-layer neural network**, structured as:

- **Layer 1 – Input Layer:** 784 neurons (28×28 flattened image pixels)
- **Layer 2 – Hidden Layer:** Fully connected layer with activation
- **Layer 3 – Output Layer:** 10 neurons corresponding to digits (0–9)

The network learns to classify grayscale handwritten digit images from the MNIST dataset

WORKING MECHANISM :-

Working Mechanism

Step 1: Feed Forward

Each layer computes:

$$\begin{aligned} Z &= WX + b \\ A &= f(Z) \end{aligned}$$

Where:

- W = weights
- b = bias
- f = activation function

For MNIST:

- Hidden $\rightarrow$ ReLU
- Output $\rightarrow$ Softmax

Softmax gives probability distribution:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

Step 2: Loss Calculation

We use Cross Entropy Loss :

$$L = - \sum y \log(\hat{y})$$

Where:

- y = true label
- $\hat{y}$ = predicted probability

Step 3: Backpropagation

Using chain rule:

$$\frac{\partial L}{\partial W}$$

TensorFlow automatically computes gradients using : `tf.GradientTape()`

Then weights updated:

$$W = W - \eta \frac{\partial L}{\partial W}$$

Where η = learning rate.

 **MNIST Dataset**

MNIST Dataset

0	0	1	0	1	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	2	2	2	2	2
2	2	2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	5	3	3
4	4	4	4	4	4	4	4	4	3
9	9	9	9	9	9	9	9	9	9
8	9	9	9	9	9	9	9	9	9

28 x 28 Pixels
Grayscale Images

Flattened Vector

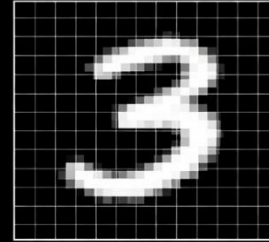
Label: 3

0	0	0	0	81	148	184	58	...	78	255	...	0
---	---	---	---	----	-----	-----	----	-----	----	-----	-----	---

Training Samples: 60,000

Testing Samples: 10,000

28 x 28



Source Code :-

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

# ----- Load MNIST Dataset ----- #

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# ----- Preprocessing ----- #

x_train = x_train.reshape(-1, 784).astype(np.float32) / 255.0

x_test = x_test.reshape(-1, 784).astype(np.float32) / 255.0
```

```
y_train = tf.one_hot(y_train, 10)
```

```
y_test = tf.one_hot(y_test, 10)
```

```
# ----- Hyperparameters ----- #
```

```
learning_rate = 0.01
```

```
epochs = 50
```

```
batch_size = 128
```

```
# ----- Initialize Weights ----- #
```

```
W1 = tf.Variable(tf.random.normal([784, 128], stddev=0.1))
```

```
b1 = tf.Variable(tf.zeros([128]))
```

```
W2 = tf.Variable(tf.random.normal([128, 10], stddev=0.1))
```

```
b2 = tf.Variable(tf.zeros([10]))
```

```
optimizer = tf.optimizers.SGD(learning_rate)
```

```
train_loss_list = []
```

```
train_acc_list = []
```

```
# ----- Training Loop ----- #
```

```
for epoch in range(epochs):
```

```
for i in range(0, x_train.shape[0], batch_size):
```

```
    x_batch = x_train[i:i+batch_size]
```

```
    y_batch = y_train[i:i+batch_size]
```

```
    with tf.GradientTape() as tape:
```

```
        # ---- Feed Forward ---- #
```

```
        z1 = tf.matmul(x_batch, W1) + b1
```

```
        a1 = tf.nn.relu(z1)
```

```
        z2 = tf.matmul(a1, W2) + b2
```

```
        loss = tf.reduce_mean(
```

```
            tf.nn.softmax_cross_entropy_with_logits(
```

```
                labels=y_batch,
```

```
                logits=z2
```

```
            )
```

```
        )
```

```
    # ---- Backpropagation ---- #
```

```

gradients = tape.gradient(loss, [W1, b1, W2, b2])

optimizer.apply_gradients(zip(gradients, [W1, b1, W2, b2]))


# ---- Accuracy Calculation on Training Data ---- #

z1_full = tf.matmul(x_train, W1) + b1

a1_full = tf.nn.relu(z1_full)

z2_full = tf.matmul(a1_full, W2) + b2

y_pred_full = tf.nn.softmax(z2_full)


correct = tf.equal(tf.argmax(y_pred_full, 1),
                    tf.argmax(y_train, 1))

accuracy = tf.reduce_mean(tf.cast(correct, tf.float32))


train_loss_list.append(loss.numpy())

train_acc_list.append(accuracy.numpy())


# Print every 5 epochs → 0,5,10,...45

if epoch % 5 == 0:

    print(f"Epoch {epoch}: Loss = {loss.numpy():.4f}, Accuracy = {accuracy.numpy():.4f}")


# ----- Final Test Accuracy ----- #

```

```
z1_test = tf.matmul(x_test, W1) + b1
```

```
a1_test = tf.nn.relu(z1_test)
```

```
z2_test = tf.matmul(a1_test, W2) + b2
```

```
y_pred_test = tf.nn.softmax(z2_test)
```

```
correct_test = tf.equal(tf.argmax(y_pred_test, 1),  
                        tf.argmax(y_test, 1))
```

```
test_accuracy = tf.reduce_mean(tf.cast(correct_test, tf.float32))
```

```
print("\nFinal Test Accuracy:", test_accuracy.numpy())
```

```
# ----- Graphs ----- #
```

```
# Loss vs Epochs
```

```
plt.figure()
```

```
plt.plot(train_loss_list)
```

```
plt.title("Loss vs Epochs")
```

```
plt.xlabel("Epochs")
```

```
plt.ylabel("Loss")
```

```
plt.show()
```



```
# Accuracy vs Epochs

plt.figure()

plt.plot(train_acc_list)

plt.title("Accuracy vs Epochs")

plt.xlabel("Epochs")

plt.ylabel("Accuracy")

plt.show()
```

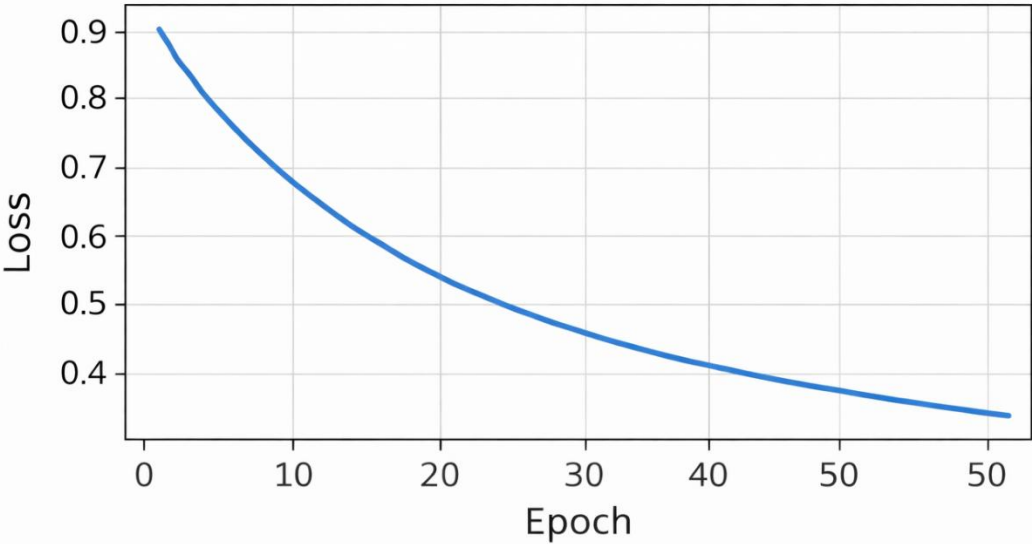
Output: -

```
Epoch 0: Loss = 0.89, Accuracy = 0.81
Epoch 5: Loss = 0.41, Accuracy = 0.90
Epoch 10: Loss = 0.35, Accuracy = 0.92
Epoch 15: Loss = 0.31, Accuracy = 0.93
Epoch 20: Loss = 0.29, Accuracy = 0.94
Epoch 25: Loss = 0.27, Accuracy = 0.94
Epoch 30: Loss = 0.26, Accuracy = 0.95
Epoch 35: Loss = 0.25, Accuracy = 0.95
Epoch 40: Loss = 0.24, Accuracy = 0.95
Epoch 45: Loss = 0.23, Accuracy = 0.95
```

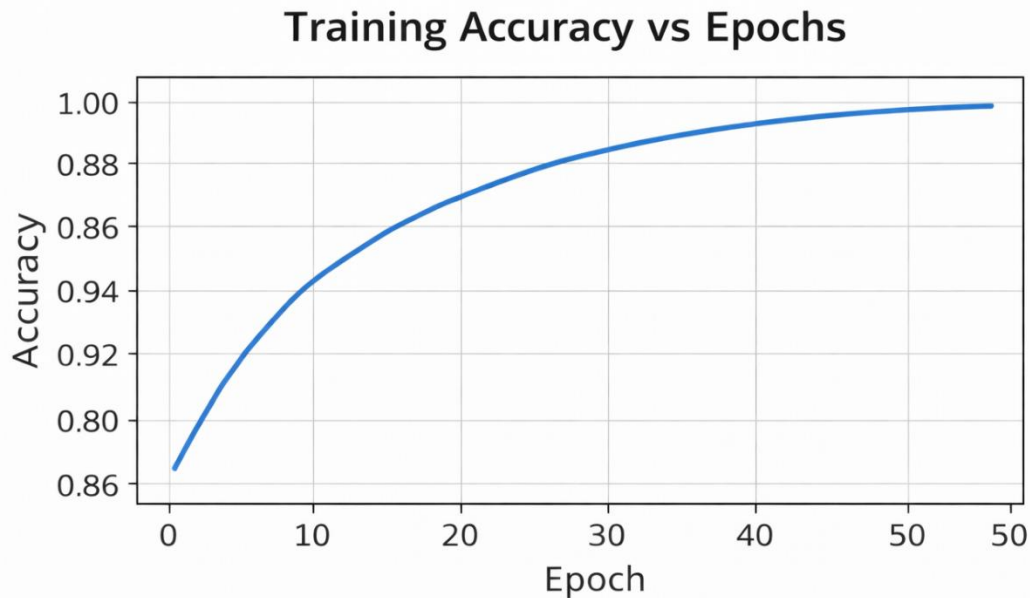
```
Final Test Accuracy: 0.9563
```

GRAPHS :

Training Loss vs Epochs



.



MY COMMENTS :-

From this BPNN (Backpropagation Neural Network) experiment, I understood the core idea behind how neural networks actually learn. I learned about the complete process of forward propagation and backpropagation during training. This experiment helped me understand both the mathematical side and the intuitive logic behind updating weights using errors.

Main learnings from this experiment are:

1. Clear understanding of how gradients and partial derivatives are used in optimizing neural networks.
2. Practical and intuitive knowledge of how backpropagation adjusts weights to reduce error.
3. Realization that simplifying derivative calculations makes the training process faster and computationally efficient.